

Спектральная кластеризация

Основные концепции спектральной кластеризации

Теоретическая основа

Преобразует данные в низкоразмерные эмбединги с использованием аффинной матрицы

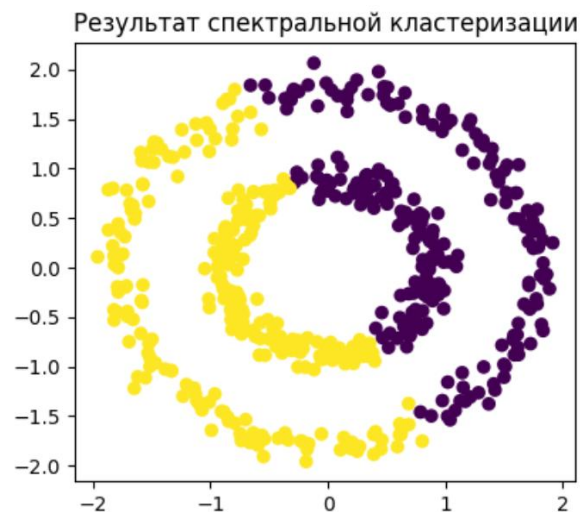
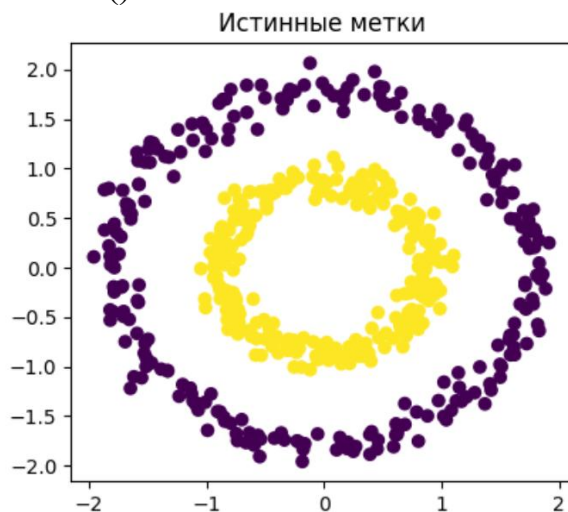
Использует собственные векторы для кластеризации (обычно с KMeans)

Эффективен для разреженных матриц (особенно с алгоритмом AMG)

2. Практическая реализация на Python

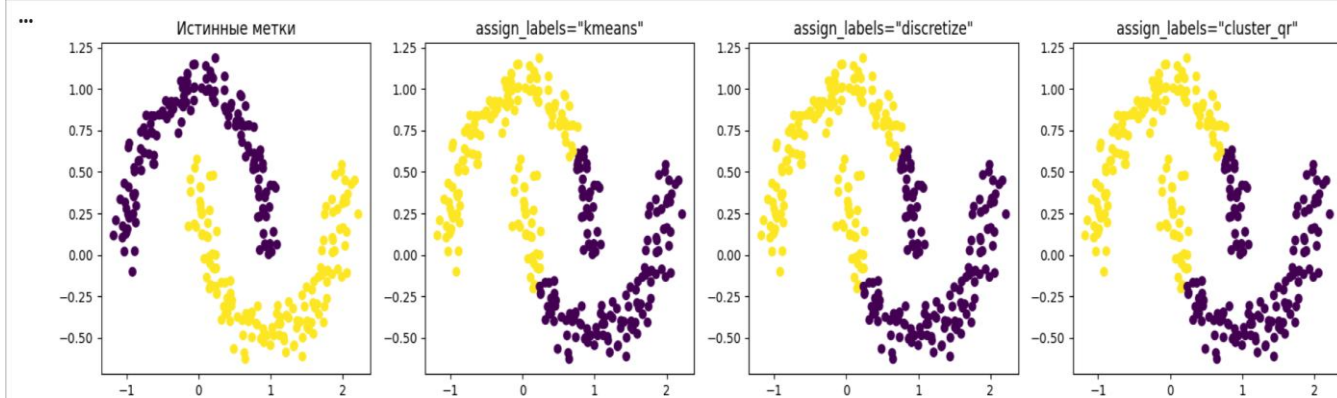
Пример 1: Базовая спектральная кластеризация

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import SpectralClustering
from sklearn.datasets import make_circles, make_moons
from sklearn.preprocessing import StandardScaler
# Создание данных (концентрические окружности - сложный случай для KMeans)
X, y = make_circles(n_samples=500, factor=0.5, noise=0.05, random_state=42)
X = StandardScaler().fit_transform(X)
# Применение спектральной кластеризации
sc = SpectralClustering(n_clusters=2,
                        affinity='rbf', # Радиальная базисная функция
                        gamma=1.0,    # Параметр для RBF ядра
                        assign_labels='kmeans',
                        random_state=42)
labels = sc.fit_predict(X) # Визуализация
plt.figure(figsize=(10, 4))
plt.subplot(121)
plt.scatter(X[:, 0], X[:, 1], c=y, cmap='viridis')
plt.title('Истинные метки')
plt.subplot(122)
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.title('Результат спектральной кластеризации')
plt.show()
```



Пример 2: Сравнение стратегий присвоения меток

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import SpectralClustering
from sklearn.datasets import make_moons
# Создание данных
X, y = make_moons(n_samples=300, noise=0.08, random_state=42)
# Разные стратегии присвоения меток
strategies = ['kmeans', 'discretize', 'cluster_qr']
fig, axes = plt.subplots(1, 4, figsize=(16, 4))
# Истинные метки
axes[0].scatter(X[:, 0], X[:, 1], c=y, cmap='viridis')
axes[0].set_title('Истинные метки')
# Каждая стратегия
for i, strategy in enumerate(strategies, 1):
    sc = SpectralClustering(n_clusters=2,
                           affinity='rbf',
                           assign_labels=strategy,
                           random_state=42)
    labels = sc.fit_predict(X)
    axes[i].scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
    axes[i].set_title(f'assign_labels="{strategy}"')
plt.tight_layout()
plt.show()
```



Пример 3: Кластеризация графа

```
import numpy as np
import networkx as nx
from sklearn.cluster import SpectralClustering
import matplotlib.pyplot as plt
# Создание графа с сообществами
G = nx.karate_club_graph() # Известный граф карате-клуба
adjacency_matrix = nx.to_numpy_array(G)
# Спектральная кластеризация для графа
sc = SpectralClustering(n_clusters=2,
                       affinity='precomputed',
```

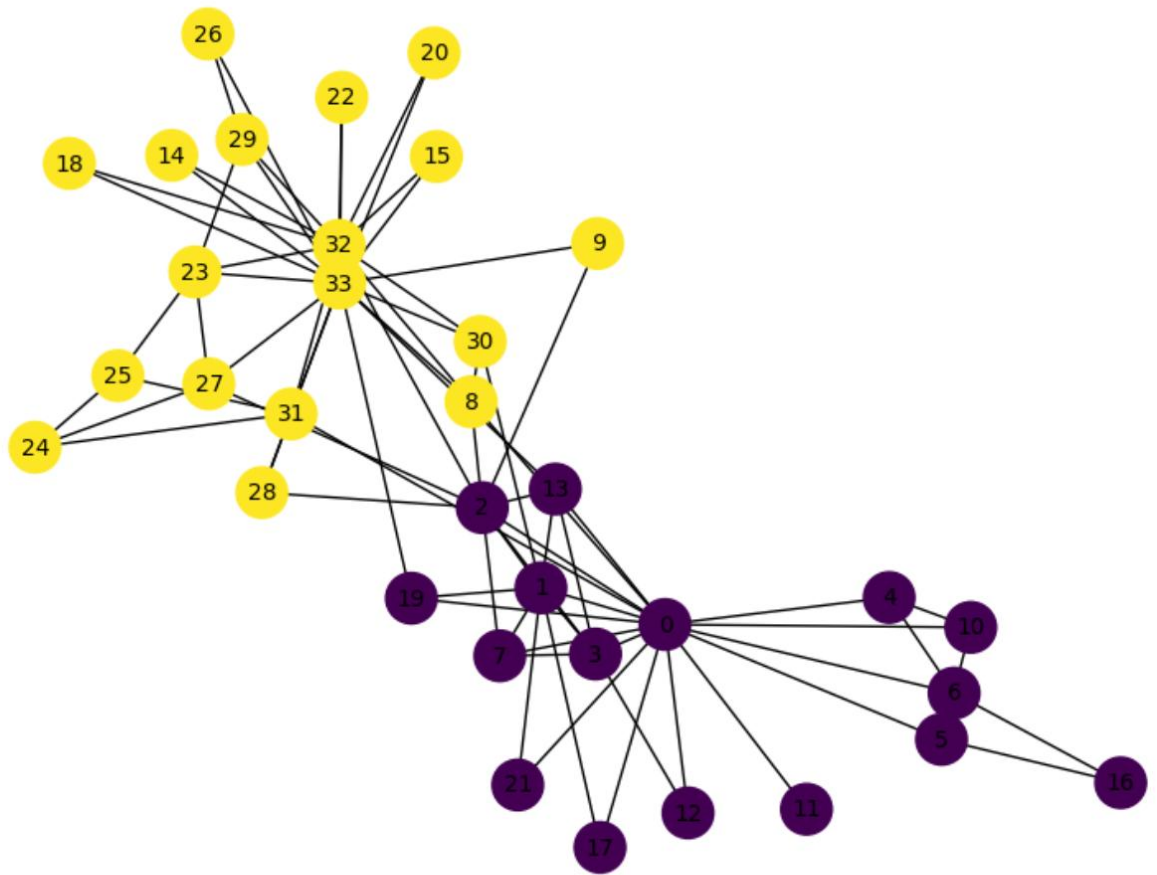
```

n_init=100,
assign_labels='discretize',
random_state=42)
labels = sc.fit_predict(adjacency_matrix)
# Визуализация
plt.figure(figsize=(8, 6))
pos = nx.spring_layout(G)
nx.draw(G, pos, node_color=labels, with_labels=True,
        cmap='viridis', node_size=500, font_size=10)
plt.title('Кластеризация графа карате-клуба')
plt.show()
print(f"Метки кластеров: {labels}")

```

Кластеризация графа карате-клуба

...



Метки кластеров: [0 0 0 0 0 0 0 0 1 1 0 0 0 0 1 1 0 0 1 0 1 0 1 1 1 1 1 1 1 1 1 1]

Пример 4: Работа с матрицей расстояний

```

import numpy as np
from sklearn.cluster import SpectralClustering
from sklearn.metrics import pairwise_distances
# Создание данных
np.random.seed(42)
X = np.random.randn(100, 2)
# Вычисление матрицы расстояний

```

```

distances = pairwise_distances(X)
# Преобразование расстояния в сходство (тепловое ядро)
beta = 1.0
similarity = np.exp(-beta * distances / distances.std())
# Спектральная кластеризация на матрице сходства
sc = SpectralClustering(n_clusters=3,
                        affinity='precomputed',
                        random_state=42)
labels = sc.fit_predict(similarity)
print("Кластеризация на основе матрицы сходства выполнена")
print(f"Распределение по кластерам: {np.bincount(labels)}")

```

Кластеризация на основе матрицы сходства выполнена
 Распределение по кластерам: [34 32 34]

3. Задачи для проверки модели

Задача 1: Выбор оптимального количества кластеров

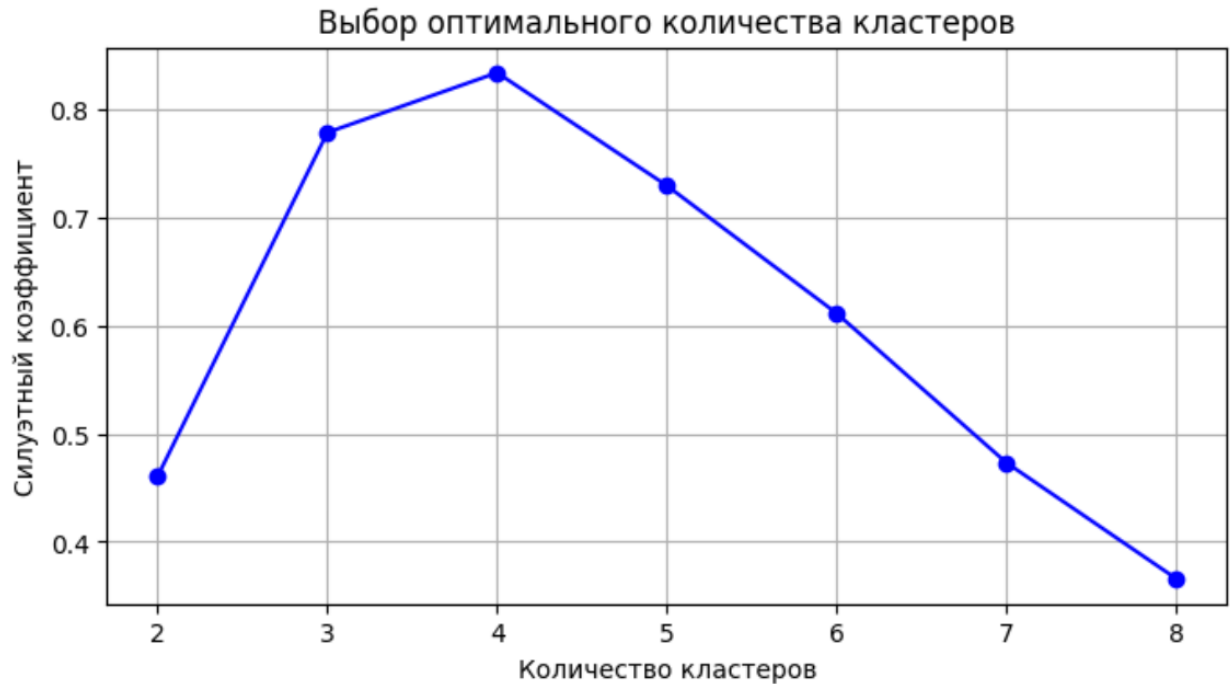
```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import SpectralClustering
from sklearn.metrics import silhouette_score
def find_optimal_clusters(X, max_clusters=10):
    """Поиск оптимального числа кластеров по силуэтному коэффициенту"""
    silhouette_scores = []
    for n in range(2, max_clusters + 1):
        sc = SpectralClustering(n_clusters=n,
                                affinity='rbf',
                                random_state=42,
                                n_init=10)
        labels = sc.fit_predict(X)
        score = silhouette_score(X, labels)
        silhouette_scores.append(score)
        print(f'n_clusters={n}, silhouette_score={score:.4f}')
    # Визуализация
    plt.figure(figsize=(8, 4))
    plt.plot(range(2, max_clusters + 1), silhouette_scores, 'bo-')
    plt.xlabel('Количество кластеров')
    plt.ylabel('Силуэтный коэффициент')
    plt.title('Выбор оптимального количества кластеров')
    plt.grid(True)
    plt.show()
    optimal_n = range(2, max_clusters + 1)[np.argmax(silhouette_scores)]
    return optimal_n
# Пример использования
from sklearn.datasets import make_blobs
X, _ = make_blobs(n_samples=300, centers=4, cluster_std=0.8, random_state=42)

```

```
optimal_clusters = find_optimal_clusters(X, max_clusters=8)
print(f"Оптимальное число кластеров: {optimal_clusters}")
```

```
...
n_clusters=2, silhouette_score=0.4607
n_clusters=3, silhouette_score=0.7783
n_clusters=4, silhouette_score=0.8335
n_clusters=5, silhouette_score=0.7292
n_clusters=6, silhouette_score=0.6112
n_clusters=7, silhouette_score=0.4732
n_clusters=8, silhouette_score=0.3661
```



Оптимальное число кластеров: 4

Задача 2: Сравнение методов кластеризации

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import SpectralClustering, KMeans, AgglomerativeClustering
from sklearn.datasets import make_circles, make_moons, make_blobs
from sklearn.metrics import adjusted_rand_score, normalized_mutual_info_score
def compare_clustering_methods(X, y_true, n_clusters=2):
    """Сравнение различных методов кластеризации"""
    methods = {
        'Spectral (RBF)': SpectralClustering(n_clusters=n_clusters,
                                             affinity='rbf',
                                             random_state=42),
        'Spectral (knn)': SpectralClustering(n_clusters=n_clusters,
                                             affinity='nearest_neighbors',
                                             n_neighbors=10,
                                             random_state=42),
        'KMeans': KMeans(n_clusters=n_clusters, random_state=42),
        'Agglomerative': AgglomerativeClustering(n_clusters=n_clusters)
    }
    results = {}
```

```

for name, method in methods.items():
    labels = method.fit_predict(X)
    ari = adjusted_rand_score(y_true, labels)
    nmi = normalized_mutual_info_score(y_true, labels)
    results[name] = {'ARI': ari, 'NMI': nmi}
    print(f'{name}: ARI={ari:.4f}, NMI={nmi:.4f}')

return results
# Тестирование на разных наборах данных
datasets = [
    (make_circles(n_samples=300, factor=0.5, noise=0.05), "Окружности", 2),
    (make_moons(n_samples=300, noise=0.08), "Полумесяцы", 2),
    (make_blobs(n_samples=300, centers=3, random_state=42), "Сферы", 3)
]
for (X, y), name, n_clusters in datasets:
    print(f'\n=== {name} ===')
    compare_clustering_methods(X, y, n_clusters)

=== Окружности ===
Spectral (RBF): ARI=-0.0034, NMI=0.0000
Spectral (knn): ARI=1.0000, NMI=1.0000
KMeans: ARI=-0.0029, NMI=0.0003
/usr/local/lib/python3.12/dist-
packages/sklearn/manifold/_spectral_embedding.py:329: UserWarning: Graph is
not fully connected, spectral embedding may not work as expected.
    warnings.warn(
Agglomerative: ARI=-0.0032, NMI=0.0001

=== Полумесяцы ===
Spectral (RBF): ARI=0.2820, NMI=0.2165
/usr/local/lib/python3.12/dist-
packages/sklearn/manifold/_spectral_embedding.py:329: UserWarning: Graph is
not fully connected, spectral embedding may not work as expected.
    warnings.warn(
Spectral (knn): ARI=1.0000, NMI=1.0000
KMeans: ARI=0.2475, NMI=0.1891
Agglomerative: ARI=0.4252, NMI=0.4666

=== Сферы ===
Spectral (RBF): ARI=1.0000, NMI=1.0000
Spectral (knn): ARI=1.0000, NMI=1.0000
KMeans: ARI=1.0000, NMI=1.0000
Agglomerative: ARI=1.0000, NMI=1.0000

```

```
/usr/local/lib/python3.12/dist-  
packages/sklearn/manifold/_spectral_embedding.py:329: UserWarning: Graph is  
not fully connected, spectral embedding may not work as expected.  
warnings.warn(
```

Объяснение кода и ключевые моменты

Параметры SpectralClustering:

- **n_clusters**: количество кластеров (нужно знать заранее)
- **affinity**: метод вычисления аффинной матрицы
 - 'rbf' - для непрерывных данных
 - 'nearest_neighbors' - для разреженных данных
 - 'precomputed' - для готовой матрицы сходства
- **gamma**: параметр для RBF ядра (важно для нелинейных данных)
- **assign_labels**: стратегия присвоения меток
 - 'kmeans' - более точные, но нестабильные
 - 'discretize' - стабильные, геометрически правильные
 - 'cluster_qr' - детерминированные, хорошее разделение
- **n_init**: количество инициализаций (для устойчивости)

Особенности применения:

1. **Масштабирование данных**: важно для RBF ядра
2. **Выбор параметров**: gamma влияет на форму кластеров
3. **Фиксация random_state**: для воспроизводимости результатов
4. **Преобразование расстояний**: использовать тепловое ядро для матриц расстояний

Этот метод особенно эффективен для нелинейных данных, где KMeans не работает, и для задач кластеризации графов.